

Workflow

peanutixx

2026 年 4 月 12 日

Contents

1 简介	2
1.1 安装	2
1.2 快速开始	2
1.3 Workflow 设计思想	3
2 HTTP 客户端	4
2.1 示例：获取百度首页	4
2.2 讲解	5
3 MySQL 任务	8
3.1 示例：DML 操作	8
3.2 示例：DQL 操作	10
3.3 讲解	11
4 SeriesWork：串行任务流	15
4.1 示例：抓取网页，将网页元数据存入 MySQL	15
4.2 讲解	18
5 HTTP 服务器	20
5.1 示例：echo 服务器	20
5.2 讲解	22
6 磁盘 I/O 任务	23
6.1 示例：静态资源服务器	23
6.2 讲解	25
7 定时器任务	26
7.1 示例：周期性执行任务	26
7.2 讲解	27
8 计算任务	28
8.1 示例：WFGoTask	28
8.2 讲解	29
9 DAG 图任务（了解）	30
9.1 示例：WFGraphTask	30
9.2 讲解	32
10 ParallelWork：并行任务流	33
10.1 示例：并行执行两个序列	33
10.2 示例：并行抓取网页，并按序输出	35

1 简介

Workflow 是搜狗公司开发的一个 C++ 服务器引擎，它支撑着搜狗公司几乎所有的 C++ 后端服务，包括所有搜索服务，云输入法，在线广告等。Workflow 需要每日处理数百亿的请求。Workflow 是一个设计轻盈的优雅的企业级程序引擎，可以满足绝大多数后端与嵌入式开发需求。

1.1 安装

```

1 # 进入项目源代码目录
2 cd workflow
3 # 创建并进入构建目录（用于存放编译中间文件和结果）
4 mkdir build && cd build
5 # 运行 CMake 配置项目，CMakeLists.txt 在上一级目录(..)
6 cmake ..
7 # 编译项目，生成可执行文件和库文件
8 make
9 # 将编译好的文件安装到系统目录（如 /usr/local/include、/usr/local/lib）
10 sudo make install
11 # 更新动态链接库缓存，使系统能找到新安装的库文件
12 sudo ldconfig

```

安装成功后，/usr/local/include 目录下会多一个 workflow 文件夹，/usr/local/lib 目录下会多 libworkflow.a、libworkflow.so、libworkflow.so.0、libworkflow.so.0.11.9 四个文件。

1.2 快速开始

我们可以使用 Workflow 快速搭建一个 HTTP 服务器：

```

                                01_hello_workflow.cc
1 #include <iostream>
2 #include <workflow/WFHttpServer.h>
3
4 using namespace std;
5 using namespace protocol;
6
7 int main()
8 {
9     // 1. 创建 HTTP 服务器
10    WFHttpServer server([](WFHttpTask* task) {
11        // 获取 HTTP 任务（网络任务）的响应
12        HttpResponse* resp = task->get_resp();
13        // 往响应报文中追加数据
14        resp->append_output_body("<html>Hello World!</html>");
15    });
16    // 2. 启动服务器（绑定通配符地址），监听 8888 端口
17    if (server.start(8888) == 0) {
18        getchar(); // 按任意键退出
19        server.stop(); // 优雅退出

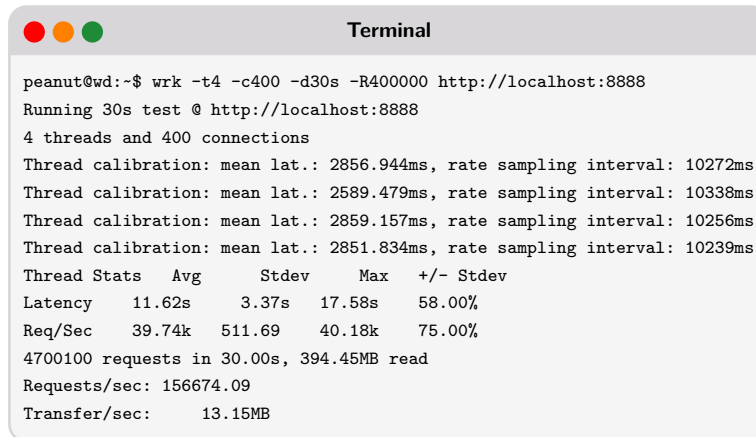
```

```

20     } else {
21         cerr << "ERROR: Server start failed!" << endl;
22         exit(1);
23     }
24
25     return 0;
26 }

```

接下来，我们使用 wrk2 对 Workflow 进行性能测试，看看在我们的机器上，吞吐量大概是多少：



```

Terminal

peanut@wd:~$ wrk -t4 -c400 -d30s -R400000 http://localhost:8888
Running 30s test @ http://localhost:8888
4 threads and 400 connections
Thread calibration: mean lat.: 2856.944ms, rate sampling interval: 10272ms
Thread calibration: mean lat.: 2589.479ms, rate sampling interval: 10338ms
Thread calibration: mean lat.: 2859.157ms, rate sampling interval: 10256ms
Thread calibration: mean lat.: 2851.834ms, rate sampling interval: 10239ms
Thread Stats   Avg      Stdev     Max    +/-  Stdev
Latency    11.62s    3.37s   17.58s   58.00%
Req/Sec    39.74k    511.69   40.18k   75.00%
4700100 requests in 30.00s, 394.45MB read
Requests/sec: 156674.09
Transfer/sec:    13.15MB

```

这个命令的含义是：

- **-t4**：启动 4 个线程。
- **-c400**：维持 400 个并发连接。
- **-d30s**：测试持续 30 秒。
- **-R400000**：目标吞吐量是每秒 400000 个请求。
- **--latency**：输出详细的延迟统计结果。

从上面的测试结果可以看出：在我的机器上，Workflow 的最大吞吐量大概在 15w 左右。

1.3 Workflow 设计思想

1. Workflow 是一个企业级、轻量级的异步任务调度框架，强调计算与通信的融合。它通过“任务流”模型，将网络、CPU、磁盘 I/O 等多种异步资源统一管理，旨在实现无损耗的资源调度。
2. Workflow 的核心思想是将一切异步资源（网络、磁盘 I/O、CPU、GPU、定时器等）都封装成任务，由框架统一调度。开发者不再需要直接操作 epoll、线程池或异步 I/O 接口，而是通过组合这些任务，以串行和并行的方式构建业务逻辑。
3. Workflow 包含五种基础任务：网络任务，计算任务，文件 I/O 任务，定时器任务，以及计数器任务。所有任务都可以由任务工厂创建，并且所有的任务都是异步执行的。
4. 所有任务都运行在某个串行流（**SeriesWork**）中，并且共享串行流的上下文（**context**）。这种机制让异步任务之间传递数据变得非常简单。
5. 任务之间可以通过串联或并联的方式构建出任务流，甚至我们可以构建更复杂的 DAG（Directed Acyclic Graph, 有向无环图）任务。
6. Workflow 有一套成熟的生命周期管理机制，大幅简化了异步程序的内存管理：
 - a) 所有任务，它们的生命周期都是从创建开始，到任务的回调函数执行结束才终止，没有内存泄漏的风险。

- b) 如果创建了任务，之后又不想执行了，则需要调用 `dismiss()` 来释放资源，否则会发生资源泄漏。
- c) 任务中包含的数据，比如网络任务中 `req`, `resp` 会随着任务的终止一同被回收。但程序员可以移动语义提前将所需的数据移走，避免大量数据的拷贝。

2 HTTP 客户端

2.1 示例：获取百度首页

我们来看一个HTTP客户端的示例：客户端请求百度服务器的首页，然后输出到终端。实现类似 `wget http://www.baidu.com -O -` 的效果。

```
02_http_client.cc
1 #include <iostream>
2 #include <string>
3 #include <workflow/HttpMessage.h>
4 #include <workflow/HttpUtil.h> // for HttpHeaderCursor
5 #include <workflow/WFFacilities.h>
6
7 using namespace std;
8 using namespace protocol;
9
10 // WaitGroup (等待组): 可以看组是条件变量和原子变量的组合。
11 WFFacilities::WaitGroup waitGroup(1);
12
13 void http_callback(WFHHttpRequest* task)
14 {
15     // 1. 检查任务状态
16     int state = task->get_state();
17     // 软件设计原则: 及早失败原则
18     // 疾在腠理, 汤熨之所及也; 在肌肤, 针石之所及也; 在肠胃, 火齐之所及也; 在骨髓, 司命之所属
19     if (state != WFT_STATE_SUCCESS) {
20         // 打印错误信息
21         cerr << WFGlobal::get_error_string(state, task->get_error()) << endl;
22         waitGroup.done(); // 任务结束, 将 waitGroup 的值减 1
23         return;
24     }
25     // 2. 任务一定成功, 将响应报文输出到终端
26     HttpResponse* resp = task->get_resp();
27     // 响应行
28     cout << resp->get_http_version() << " "
29          << resp->get_status_code() << " "
30          << resp->get_reason_phrase() << "\r\n";
31     // 响应头部
32     HttpHeaderCursor cursor(resp);
33     string name;
34     string value;
35     while (cursor.next(name, value)) {
36         cout << name << ": " << value << "\r\n";
37     }
38     cout << "\r\n";
39     // 响应体
```

```

40     const void* body;
41     size_t size;
42     resp->get_parsed_body(&body, &size);
43     cout << static_cast<const char*>(body) << endl;
44     // 任务结束, 将 waitGroup 的值减 1
45     waitGroup.done();
46 }
47
48 int main()
49 {
50     // 1. 使用静态工厂方法 WFTaskFactory::create_http_task() 创建 Http 任务
51     // url: 请求的 url
52     // redirect_max: 最大的重定向次数
53     // retry_max: 最大的重试次数
54     // callback: 回调函数, Http 任务结束后调用, 用于处理服务器的响应
55     WFHttpTask* httpTask = WFTaskFactory::create_http_task("http://www.baidu.com", 3, 2, http_callback);
56     // 2. 设置 Http 请求
57     HttpRequest* req = httpTask->get_req();
58     // 设置请求方法
59     req->set_method("GET");
60     // 设置请求 URI, 可以不设置, WFHttpTask 可以从 URL 中自动解析。
61     req->set_request_uri("/");
62     // 添加请求头
63     req->add_header_pair("Accept", "/*/*");
64     req->add_header_pair("User-Agent", "myHttpTask");
65     // 修改请求头
66     req->set_header_pair("Connection", "close");
67     // 3. 将任务交给 Workflow 异步执行
68     httpTask->start();
69
70     // 主线程阻塞, 直到 waitGroup 的值减为 0
71     waitGroup.wait();
72     return 0;
73 }

```

2.2 讲解

1. 创建 HTTP 任务

我们可以使用 WFTaskFactory 的静态工厂方法 create_http_task() 创建一个 HTTP 任务, 方法定义如下:

```

// 模板的特化
using WFHttpTask = WFNetworkTask<protocol::HttpRequest, protocol::HttpResponse>;

// 可调用对象: 函数指针, 函数对象, lambda, 可以通过 std::bind() 做参数适配
using http_callback_t = std::function<void (WFHttpTask *)>;

class WFTaskFactory
{
public:
    static WFHttpTask *create_http_task(const std::string& url,           // URL

```

```

        int redirect_max,           // 最大的重定向次数
        int retry_max,             // 最大的重试次数
        http_callback_t callback);  // 回调函数
    // ...
};

```

回调函数 `callback` 可以设置为 `nullptr`，表示任务结束后，直接终止，不需要额外处理。所有任务的回调函数都遵循这个风格。哪怕传入 `url` 不合法，`WFTaskFactory` 的静态工厂方法也不会失败，所以不用担心返回的 `task` 为空指针。一切错误都是在回调函数中处理。

2. 设置请求

我们可以通过 `httpTask->get_req()` 获取 HTTP 任务的请求 `HttpRequest`。`HttpRequest` 默认使用 `GET` 方法，`HTTP/1.1`，长连接。框架会根据传入的 `url` 自动设置 `request_uri`，`Host` 等。框架会在给服务器发送请求前，会根据需要自动添加 `Content-Length` 和 `Connection` 这些头部。当然我们也可以通过 `add_header_pair()` 方法添加自己想要的头部。

```

class HttpRequest : public ProtocolMessage
{
public:
    const char *get_http_version() const;
    bool set_http_version(const char *version);
    bool is_chunked() const;
    bool is_keep_alive() const;
    bool add_header(const struct HttpMessageHeader *header);
    bool add_header_pair(const char *name, const char *value);
    bool set_header(const struct HttpMessageHeader *header);
    bool set_header_pair(const char *name, const char *value);
    bool get_parsed_body(const void **body, size_t *size) const;
    bool append_output_body(const void *buf, size_t size);
    bool append_output_body(const char *buf);
    bool append_output_body_nocopy(const void *buf, size_t size);
    bool append_output_body_nocopy(const char *buf);
    void clear_output_body();
    // ...

public:
    bool get_http_version(std::string& version) const;
    bool set_http_version(const std::string& version);
    bool add_header_pair(const std::string& name, const std::string& value);
    bool set_header_pair(const std::string& name, const std::string& value);
    bool append_output_body(const std::string& buf);
    bool append_output_body_nocopy(const std::string& buf);
    // ...
};

class HttpRequest : public HttpMessage
{
public:
    const char *get_method() const;
    const char *get_request_uri() const;
    bool set_method(const char *method);
    bool set_request_uri(const char *uri);
    //...
};

```

```

public:
    bool get_method(std::string& method) const;
    bool get_request_uri(std::string& uri) const;
    bool set_method(const std::string& method);
    bool set_request_uri(const std::string& uri);
    //...
};

```

3. 启动任务

我们可以通过 `httpTask->start()` 来启动任务。这个方法是非阻塞的，并且不会失败。也就是说，之后必然会调用任务的回调函数（回顾：所有的错误都是在任务的回调函数中处理）。

任务是异步执行的，任务启动之后，直到回调函数调用之前，我们都不能再使用 `httpTask` 指针。换句话说就是，任务启动之后，我们就只能在回调函数中再操作任务指针了。由于 `start()` 是非阻塞的，我们启动任务之后，使用 `waitGroup.wait()` 让主线程阻塞，直到任务的回调函数结束（`waitGroup` 的值才会减 1）。

```

WFTask.h
template<class REQ, class RESP>
class WFNetworkTask : public CommRequest
{
public:
    void start();    // 启动任务
    void dismiss();  // 取消任务
public:
    REQ *get_req() { return &this->req; }    // 获取请求
    RESP *get_resp() { return &this->resp; }  // 获取响应
public:
    void *user_data;    // 任务可以携带用户自定义数据
public:
    int get_state() const { return this->state; }    // 获取任务的状态
    int get_error() const { return this->error; }    // 获取任务的错误码
    int get_peer_addr(struct sockaddr *addr, socklen_t *addrlen) const;    // 获取对端的套接字地址
    // ...
public:
    // 设置任务的预处理函数（任务开始前执行，一般不使用）
    void set_prepare(std::function<void (WFNetworkTask<REQ, RESP> *)> prep);
    // 设置任务的回调函数（任务完成时执行）
    void set_callback(std::function<void (WFNetworkTask<REQ, RESP> *)> cb);
};

```

4. 处理响应

`task->get_state()` 和 `task->get_error()` 可以分别获取任务的状态码和错误码。如果任务发生了错误，还可以通过 `WFGlobal::get_error_string(state, task->get_error())` 来获取具体的错误原因。

`task->get_resp()` 可以获取响应报文 `HttpResponse`，这个和 `HttpRequest` 区别不大，都是 `HttpMessage` 的派生类。

```

WFTask.h
class HttpResponse : public HttpMessage
{
public:
    const char *get_status_code() const
    const char *get_reason_phrase() const
    bool set_status_code(const char *code)
    bool set_reason_phrase(const char *phrase)

```

```

    // ...
};

```

我们还可以通过 `HttpHeaderCursor` 对象，解析 `HttpRequest` 和 `HttpResponse` 的头部字段。

```

HttpUtil.h
class HttpHeaderCursor
{
public:
    HttpHeaderCursor(const HttpMessage *message);    // 需要传入请求报文或响应报文
    virtual ~HttpHeaderCursor();
public:
    bool next(std::string& name, std::string& value);    // 获取下一个头部字段
    bool find(const std::string& name, std::string& value);    // 获取某个头部字段的值
    void rewind();    // 回到头部的起始位置
    // ...
};

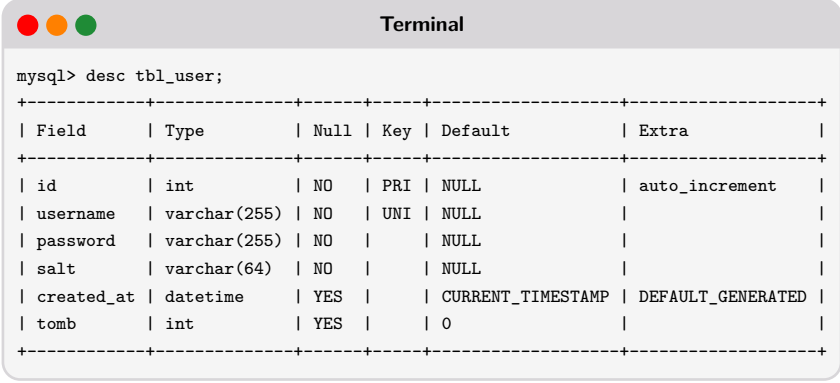
```

`resp->get_parsed_body(&body, &size)` 可以获取响应体，当 `httpTask` 成功时，该函数必然会返回 `true`。这个函数不会拷贝响应体，而是将 `body` 指针指向 `HttpResponse` 的数据区。

3 MySQL 任务

3.1 示例：DML 操作

我们来看一个 MySQL 任务的例子：往 `127.0.0.1/demo` 数据库的 `tbl_user` 表中插入一条记录。`tbl_user` 表的结构如下：



```

mysql> desc tbl_user;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int  | NO   | PRI | NULL    | auto_increment |
| username | varchar(255) | NO | UNI | NULL    |
| password | varchar(255) | NO |     | NULL    |
| salt   | varchar(64) | NO |     | NULL    |
| created_at | datetime | YES |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| tomb   | int  | YES |     | 0       |
+-----+-----+-----+-----+-----+-----+

```

`tbl_user` 表的建表语句：

```

CREATE TABLE tbl_user(
id int PRIMARY KEY AUTO_INCREMENT,
username varchar(255) NOT NULL UNIQUE,
password varchar(255) NOT NULL,
salt varchar(64) NOT NULL,
created_at datetime DEFAULT CURRENT_TIMESTAMP(),
tomb int DEFAULT 0
);

```

```

03_mysql_insert.cc
1 #include <iostream>
2 #include <string>

```



```

3 #include <workflow/MySQLResult.h>
4 #include <workflow/WFFacilities.h>
5 #include <workflow/WFTaskFactory.h>
6 #include <workflow/Workflow.h>
7
8 using namespace std;
9 using namespace protocol;
10
11 void mysql_callback(WFMySQLTask* task)
12 {
13     // 1. 判断任务是否成功
14     int state = task->get_state();
15     if (state != WFT_STATE_SUCCESS) {
16         cerr << WFGlobal::get_error_string(state, task->get_error()) << endl;
17         return;
18     }
19     // 2. 判断返回包的类型
20     MySQLResponse* resp = task->get_resp();
21     if (resp->get_packet_type() == MYSQL_PACKET_ERROR) {
22         cerr << "error_code: " << resp->get_error_code()
23             << ", error_msg: " << resp->get_error_msg() << endl;
24         return;
25     }
26     // 3. 处理结果集
27     MySQLResultCursor cursor(resp);
28     if (cursor.get_cursor_status() == MYSQL_STATUS_OK) { // DML 操作成功
29         unsigned long long rows = cursor.get_affected_rows(); // 获取受影响的行数
30         cout << rows << " rows affected" << endl;
31         unsigned long long id = cursor.get_insert_id(); // 获取插入记录的 id
32         cout << "insert id: " << id << endl;
33     }
34 }
35
36 int main()
37 {
38     // 1. 创建任务
39     string url = "mysql://root:1234@127.0.0.1/demo";
40     int retry_max = 3;
41     WFMySQLTask* task = WFTaskFactory::create_mysql_task(url, retry_max, mysql_callback);
42     // 2. 设置任务: 指定 SQL 语句
43     MySQLRequest* req = task->get_req();
44     string sql = "INSERT INTO tbl_user (username, password, salt) VALUES ('peanut', '1234', 'RAND_SALT')";
45     req->set_query(sql);
46     // 3. 添加到任务序列（串行流）中
47     WFFacilities::WaitGroup waitGroup(1);
48     SeriesWork* series = Workflow::create_series_work(task, [&waitGroup](const SeriesWork*) {
49         waitGroup.done();
50     });
51     // 4. 启动序列
52     series->start();
53     // 5. 主线程等待整个序列执行完毕

```

```

54     waitGroup.wait();
55     return 0;
56 }

```

3.2 示例: DQL 操作

上个例子演示的是 DML 操作, 接下来我们来看一个 DQL 操作的例子: 查询 `tbl_user` 表中的所有记录。

```

                                04_mysql_select.cc
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <workflow/MySQLResult.h>
5  #include <workflow/WFFacilities.h>
6  #include <workflow/WFTaskFactory.h>
7  #include <workflow/Workflow.h>
8
9  using namespace std;
10 using namespace protocol;
11
12 // MySQL 数据类型和 C++ 数据类型的适配
13 void display_mysql_cell(const MySQLCell& cell)
14 {
15     // 将丑陋的代码封装起来, 避免影响其它代码
16     if (cell.is_null()) {
17         cout << "(null)";
18     } else if (cell.is_int()) {
19         cout << cell.as_int();
20     } else if (cell.is_ulonglong()) {
21         cout << cell.as_ulonglong();
22     } else if (cell.is_float()) {
23         cout << cell.as_float();
24     } else if (cell.is_double()) {
25         cout << cell.as_double();
26     } else if (cell.is_string()) {
27         cout << cell.as_string();
28     } else if (cell.is_date()) {
29         cout << cell.as_date();
30     } else if (cell.is_time()) {
31         cout << cell.as_time();
32     } else if (cell.is_datetime()) {
33         cout << cell.as_datetime();
34     }
35 }
36
37 void mysql_callback(WFMySQLTask* task)
38 {
39     // 1. 判断任务是否成功
40     int state = task->get_state();
41     if (state != WFT_STATE_SUCCESS) {
42         cerr << WFGlobal::get_error_string(state, task->get_error()) << endl;

```

```

43     return;
44 }
45 // 2. 判断 MySQL 返回包的类型
46 MySQLResponse* resp = task->get_resp();
47 if (resp->get_packet_type() == MYSQL_PACKET_ERROR) {
48     cerr << "error_code: " << resp->get_error_code()
49         << ", error_msg: " << resp->get_error_msg() << endl;
50     return;
51 }
52 // 3. 处理结果集
53 MySQLResultCursor cursor(resp);
54 if (cursor.get_cursor_status() == MYSQL_STATUS_GET_RESULT) { // DQL 操作执行成功
55     // 使用 MySQLResultCursor 遍历结果集
56     vector<MySQLCell> record;
57     while (cursor.fetch_row(record)) { // 获取下一条记录
58         for (const auto& cell : record) {
59             display_mysql_cell(cell);
60             cout << " ";
61         }
62         cout << endl;
63     }
64 }
65 }
66
67 int main()
68 {
69     // 1. 创建任务
70     string url = "mysql://root:1234@127.0.0.1/demo";
71     int retry_max = 3;
72     WFMysqlTask* task = WFTaskFactory::create_mysql_task(url, retry_max, mysql_callback);
73     // 2. 设置任务: 指定 SQL 语句
74     MySQLRequest* req = task->get_req();
75     string sql = "SELECT * FROM tbl_user";
76     req->set_query(sql);
77     // 3. 将任务添加到序列 (串行流) 中
78     WFFacilities::WaitGroup waitGroup(1);
79     SeriesWork* series = Workflow::create_series_work(task, [&waitGroup](const SeriesWork*) {
80         waitGroup.done();
81     });
82     // 4. 启动序列
83     series->start();
84     // 5. 主线程等待整个序列执行完毕
85     waitGroup.wait();
86     return 0;
87 }

```

3.3 讲解

1. 创建任务

创建 MySQL 任务与创建 HTTP 任务没有太大的区别, 只是少了 `redirect_max` 参数, 并且 URL 的格式也有所不同:

mysql://username:password@host:port/dbname?character_set=charset&character_set_results=charset

- port: 默认为 3306。
- dbname: 数据库名, 如果只需要操作一个数据库, 建议填写。
- character_set, character_set_results 一般不需要填写, 使用默认的即可。

如: mysql://root:1234@127.0.0.1/demo。

WFTaskFactory.h

```
// 模板的特化
using WFTaskFactory = WFTaskFactory<protocol::MySQLRequest, protocol::MySQLResponse>;

// 可调用对象: 函数指针, 函数对象, lambda, 可以通过 std::bind() 做参数适配
using mysql_callback_t = std::function<void (WFTaskFactory *)>;

class WFTaskFactory
{
public:
    static WFTaskFactory *create_mysql_task(const std::string& url,
        int retry_max,
        mysql_callback_t callback);
    //...
};
```

2. 设置任务

任务创建完之后, 就需要设置了。可以使用 MySQLRequest 的 set_query() 方法指定需要执行的 SQL 语句了。如果没有指定 SQL 语句就启动 MySQL 任务, 任务会执行失败, 并且错误码会设置为 WFT_ERR_MYSQL_QUERY_NOT_SET。

MySQLMessage.h

```
class MySQLMessage : public ProtocolMessage
{
public:
    MySQLMessage();
    virtual ~MySQLMessage();
    MySQLMessage(MySQLMessage&& move); // 移动构造函数
    MySQLMessage& operator= (MySQLMessage&& move); // 移动赋值函数
};

class MySQLRequest : public MySQLMessage
{
public:
    void set_query(const std::string& query); // 设置 SQL 语句
    std::string get_query() const; // 获取 SQL 语句
    bool query_is_unset() const;
};
```

MySQLTask 目前支持的 SQL 语句, 已经能涵盖基本的增删改查、建库删库、建表删表、预处理、存储过程和使用事务的需求。并且我们可以在一次请求中, 发送多条 SQL 语句:

```
req->set_query("SELECT * FROM table1; CALL procedure1(); INSERT INTO table3 (id) VALUES (1);");
```

MySQL 会按顺序依次执行 SQL 语句, 直到发生错误, 前面的 SQL 语句都执行成功。

3. 解析结果集

和别的网络任务一样，MySQLTask 可以通过 `task->get_resp()` 得到 MySQLResponse。MySQLResponse 可能包含一个或多个结果集。结果集的类型有两种，MYSQL_STATUS_OK 和 MYSQL_STATUS_GET_RESULT，分别对应 DML 和 DQL。我们可以通过 MySQLResultCursor 遍历所有的结果集。

具体解析流程：

1. 判断网络任务是否成功（有没有收到响应）

```
task->get_state() != WFT_STATE_SUCCESS
```

2. 查看最后一条 SQL 语句的返回包类型：`resp->get_packet_type()`

常见类型有：

- MYSQL_PACKET_OK: 成功，可以用 `cursor` 遍历结果集。
- MYSQL_PACKET_EOF: 成功，可以用 `cursor` 遍历结果集。
- MYSQL_PACKET_ERROR: 失败或部分失败，成功的部分可以用 `cursor` 遍历结果集。

如果返回包的类型为 MYSQL_PACKET_ERROR，可以通过 `resp->get_error_code()` 和 `resp->get_error_msg()` 获取错误码和错误信息。

3. 遍历结果集

我们可以使用 MySQLResultCursor 遍历所有的结果集。MySQLResponse 可能会包含多个结果集，`cursor` 一开始会指向第一个结果集的位置。我们可以使用下面的方法，将 `cursor` 移动到其它结果集：

```
bool next_result_set();           // 将光标移动到下一个结果集
void first_result_set();         // 将光标移动到第一个结果集
```

4. 解析结果集

首先我们得判断结果集的类型：`cursor.get_cursor_status()`，不同类型的结果集需要使用不同的方法解析：

	MYSQL_STATUS_GET_RESULT	MYSQL_STATUS_OK
SQL	SELECT	INSERT/UPDATE/DELETE/...
语义	DQL，结果集是一张二维表	DML，结果集表示操作是否成功
主要接口	<code>fetch_fields()</code> , <code>fetch_row()</code> , ...	<code>get_insert_id()</code> , <code>get_affected_rows()</code> , ...

Workflow 对应的接口（了解）：

```
MySQLMessage.h
class MySQLResponse : public MySQLMessage
{
public:
    /* 获取最后一条 SQL 语句的返回包类型：resp->get_packet_type()
     * 常见类型为：
     *     MYSQL_PACKET_OK: 成功，可以用 cursor 遍历结果；
     *     MYSQL_PACKET_EOF: 成功，可以用 cursor 遍历结果；
     *     MYSQL_PACKET_ERROR: 失败或部分失败，成功的部分可以用 cursor 遍历结果；
     */
    int get_packet_type() const;

    unsigned long long get_affected_rows() const;    // 获取受影响的行数
    unsigned long long get_last_insert_id() const;   // 获取最后一条插入记录的 id
    int get_error_code() const;                      // 获取错误码
    std::string get_error_msg() const;               // 获取错误信息
    //...
};
```

```

class MySQLCell
{
public:
    bool is_null() const;
    bool is_int() const;
    bool is_string() const;
    bool is_float() const;
    bool is_double() const;
    bool is_ulonglong() const;
    bool is_date() const;
    bool is_time() const;
    bool is_datetime() const;

public:
    // for copy
    int as_int() const;
    std::string as_string() const;
    std::string as_binary_string() const;
    float as_float() const;
    double as_double() const;
    unsigned long long as_ulonglong() const;
    std::string as_date() const;
    std::string as_time() const;
    std::string as_datetime() const;

    // for nocopy
    void get_cell_nocopy(const void **data, size_t *len, int *data_type) const;
    // ...
};

class MySQLField
{
public:
    std::string get_name() const;           // 获取字段的名字
    std::string get_table() const;          // 获取表的名字
    std::string get_db() const;             // 获取数据库的名字
    std::string get_def() const;            // 获取字段的默认值
    int get_data_type() const;              // 获取字段的数据类型
    // ...
};

class MySQLResultCursor
{
public:
    bool next_result_set();                 // 将光标移动到下一个结果集
    void first_result_set();                // 将光标移动到第一个结果集
    // 获取字段
    const MySQLField *fetch_field();
    const MySQLField *const *fetch_fields() const;
    // 获取记录 (行)
    bool fetch_row(std::vector<MySQLCell>& row_arr);

```

```

bool fetch_row(std::map<std::string, MySQLCell>& row_map);
bool fetch_row(std::unordered_map<std::string, MySQLCell>& row_map);
bool fetch_row_nocopy(const void **data, size_t *len, int *data_type);
bool fetch_all(std::vector<std::vector<MySQLCell>>& rows);

/* 获取 MySQLResultCursor 的状态
 * 常见的状态有:
 *      MYSQL_STATUS_OK:      DML 操作的结果集
 *      MYSQL_STATUS_GET_RESULT: DQL 操作的结果集
 */
int get_cursor_status() const;

int get_field_count() const;    // 获取字段个数 (DQL)
int get_rows_count() const;    // 获取记录个数 (DQL)
unsigned long long get_affected_rows() const;    // 获取受影响行数 (DML)
unsigned long long get_insert_id() const;        // 获取插入记录的 id (INSERT)

void rewind();    // 将光标移动到这一个结果集的起始位置
// ...
};

```

4 SeriesWork: 串行任务流

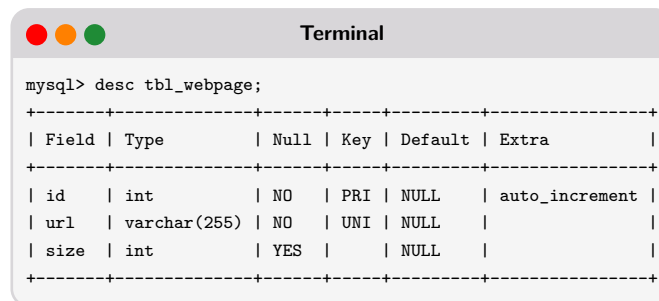
4.1 示例: 抓取网页, 将网页元数据存入 MySQL

我们通过一个例子来演示 SeriesWork (串行任务流) 的用法: 读取用户输入的 URL, 抓取网页, [解析网页], 并存入 MySQL 数据库。首先我们来创建数据库表:

```

CREATE TABLE tbl_webpage(
    id int PRIMARY KEY AUTO_INCREMENT,
    url varchar(255) NOT NULL UNIQUE,
    size int
    # 网页的其它属性
    # 网页内容一般不会存入数据库, 而是存入到文件中
);

```



```

mysql> desc tbl_webpage;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int           | NO   | PRI | NULL    | auto_increment |
| url   | varchar(255) | NO   | UNI | NULL    |                |
| size  | int           | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+

```

05_series_work.cc

```

1 #include <iostream>
2 #include <netdb.h>
3 #include <stdlib.h>
4 #include <string>
5 #include <workflow/MySQLResult.h>

```

```

6 #include <workflow/WFFacilities.h>
7 #include <workflow/WFTaskFactory.h>
8 #include <workflow/Workflow.h>
9
10 using namespace std;
11 using namespace protocol;
12
13 #define RETRY_MAX 3
14
15 struct SeriesContext {
16     string url;
17     size_t size; // 网页的大小
18     bool success; // 序列是否成功
19 };
20
21 void mysql_callback(WFMySQLTask* task)
22 {
23     // 1. 判断任务是否成功
24     int state = task->get_state();
25     if (state != WFT_STATE_SUCCESS) {
26         cerr << WFGlobal::get_error_string(state, task->get_error()) << endl;
27         return;
28     }
29     // 2. 判断返回包的类型
30     MySQLResponse* resp = task->get_resp();
31     if (resp->get_packet_type() == MYSQL_PACKET_ERROR) {
32         cerr << "error_code: " << resp->get_error_code()
33             << ", error_msg: " << resp->get_error_msg() << endl;
34         return;
35     }
36     // 3. 处理结果集
37     MySQLResultCursor cursor(resp);
38     if (cursor.get_cursor_status() == MYSQL_STATUS_OK) { // DML 操作成功
39         cout << "插入记录成功, 新纪录的 id 为: " << cursor.get_insert_id() << endl;
40         // 更改序列的上下文
41         SeriesContext* ctx = static_cast<SeriesContext*>(series_of(task)->get_context());
42         ctx->success = true;
43     }
44 }
45
46 void http_callback(WFHHttpRequestTask* task)
47 {
48     // 1. 检查任务的状态
49     int state = task->get_state();
50     if (state != WFT_STATE_SUCCESS) {
51         cerr << WFGlobal::get_error_string(state, task->get_error()) << endl;
52         return;
53     }
54     // 2. 获取任务所在的序列和序列的上下文
55     SeriesWork* series = series_of(task);
56     SeriesContext* ctx = static_cast<SeriesContext*>(series->get_context());

```



```

57 // 3. 获取响应体
58 HttpResponse* resp = task->get_resp();
59 const void* body;
60 size_t size;
61 resp->get_parsed_body(&body, &size);
62 ctx->size = size;
63 // 4. [[解析网页]]
64 cout << "网页内容: \n"
65         << static_cast<const char*>(body) << endl;
66 // 5. 创建 MySQL 任务
67 string db = "mysql://root:1234@127.0.0.1/demo";
68 WFMysqlTask* mysqlTask = WFTaskFactory::create_mysql_task(db, RETRY_MAX, mysql_callback);
69 // 设置请求
70 MySQLRequest* req = mysqlTask->get_req();
71 // 拼接 SQL 语句
72 string sql = "INSERT INTO tbl_webpage (url, size) VALUES ("
73             + ctx->url + ", "
74             + std::to_string(size) + ")";
75 cout << "[SQL]: " << sql << endl; // 打印调试信息
76 req->set_query(sql);
77 // 将 MySQL 任务添加到序列中
78 series->push_back(mysqlTask);
79 }
80
81 int main(int argc, char* argv[])
82 {
83     // 1. 校验命令行参数
84     if (argc != 2) {
85         cerr << "USAGE: " << argv[0] << " <URL>" << endl;
86         exit(1);
87     }
88     // 2. 创建 HTTP 任务
89     WFHttpTask* httpTask = WFTaskFactory::create_http_task(argv[1], 3, RETRY_MAX, http_callback);
90     // 设置请求
91     HttpRequest* req = httpTask->get_req();
92     req->add_header_pair("Accept", "/*/*");
93     req->add_header_pair("User-Agent", "wget/1.14 (linux-gnu)");
94     req->add_header_pair("Connection", "close");
95     // 设置响应的最大大小: 20M
96     httpTask->get_resp()->set_size_limit(20 * 1024 * 1024);
97     // 设置响应的超时时间: 30s
98     httpTask->set_receive_timeout(30 * 1000);
99     // 3. 创建 SeriesWork (串行任务流, 序列)
100     WFFacilities::WaitGroup waitGroup(1);
101     SeriesWork* series = Workflow::create_series_work(httpTask, [&waitGroup](const SeriesWork* series) {
102         // 获取序列的上下文
103         SeriesContext* ctx = static_cast<SeriesContext*>(series->get_context());
104         if (ctx->success) {
105             cout << "SeriesWork SUCCESS!" << endl;
106         } else {
107             cerr << "SeriesWork FAILURE!" << endl;

```

```

108     }
109     // 整个序列结束
110     delete ctx; // 释放上下文
111     waitGroup.done();
112 });
113 // 4. 设置序列的上下文
114 SeriesContext* ctx = new SeriesContext { argv[1], 0, false };
115 series->set_context(ctx);
116 // 5. 启动序列
117 series->start();
118 // 6. 主线程等待序列结束
119 waitGroup.wait();
120 return 0;
121 }

```

4.2 讲解

1. 创建 HTTP 任务并设置参数

```

WFHttpTask* httpTask = WFTaskFactory::create_http_task(...);
...
httpTask->get_resp()->set_size_limit(20 * 1024 * 1024);
httpTask->set_receive_timeout(30 * 1000);

```

`set_size_limit()` 是 `HttpMessage` 的方法，用于限制接收 HTTP 响应报文的大小。事实上所有协议的报文都有这个接口。`set_receive_timeout()` 用来设置接收响应报文的超时时间，单位为 ms。

2. 创建 *SeriesWork*，并启动

在之前的例子中，我们都是直接调用 `task->start()` 来启动任务。这样做，Workflow 会为 `task` 单独创建一个序列，并且将 `task` 作为这个序列的第一个任务，然后启动执行。

```

                                     WFTask.h
template<class REQ, class RESP>
class WFNetworkTask : public CommRequest
{
public:
    void start()
    {
        assert(!series_of(this));
        Workflow::start_series_work(this, nullptr);
    }
    ...
};

```

序列的流程图如下所示：

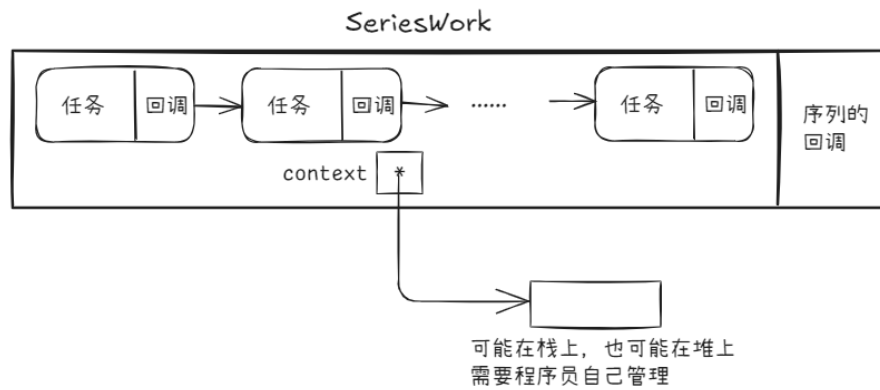


Figure 1: SeriesWork 流程图

然而在这个例子中，我们想给序列设置一个回调函数，并且添加一些上下文。所以我们不直接启动第一个任务，而是自己创建一个 `SeriesWork`，这么做显然更合理一些。在 `Workflow` 中，`SeriesWork` 表示一个串行的任务流，序列中的所有任务都能得到并修改它的上下文。序列的回调函数，会在序列中的所有任务完成之后，才开始执行。

Workflow.h

```
// 思考：为什么 SeriesWork 回调函数的参数需要用 const 修饰？
using series_callback_t = std::function<void (const SeriesWork *)>;

class Workflow
{
public:
    static SeriesWork*
    create_series_work(SubTask *first, series_callback_t callback);    // 创建序列

    static void
    start_series_work(SubTask *first, series_callback_t callback);    // 创建并启动序列

    static SeriesWork*
    create_series_work(SubTask *first, SubTask *last, series_callback_t callback);

    static void
    start_series_work(SubTask *first, SubTask *last, series_callback_t callback);
    // ...
};

class SeriesWork
{
public:
    void start();    // 启动序列：将序列的第一个任务分发给线程执行
    void dismiss();    // 创建序列后，如果不想执行，需要调用 dismiss() 释放资源
public:
    void push_back(SubTask *task);    // 往序列的末尾添加任务
    void push_front(SubTask *task);    // 往序列的开头添加任务
public:
    void *get_context() const;    // 获取序列的上下文
    void set_context(void *context);    // 设置序列的上下文
};
```

```

public:
    /* 取消一个正在运行的序列。通常，我们会在这个序列的某个任务的回调函数中调用这个方法。
     * 后续的所有任务都不会执行，并且会立刻销毁，这些任务的回调函数也不会执行。
     * 但是这个被取消序列的回调函数会执行。
     */
    virtual void cancel();
public:
    void set_callback(series_callback_t callback);
protected:
    void *context;           // 序列的上下文
    series_callback_t callback; // 序列的回调函数
    // ...
};

```

5 HTTP 服务器

5.1 示例: echo 服务器

我们即将使用 Workflow 创建一个 HTTP 服务器，它会将接收到的 HTTP 请求，以 html 格式返回给客户端。此外，它还需要完成下面这些功能：

- 程序会打印出客户端的 IP 地址，请求序号（当前 TCP 连接上的第几次请求）。
- 当一个 TCP 连接完成了 10 次请求，服务器主动关闭连接。
- 按下 **Ctrl**+**C**，程序能够正常结束，资源都会被回收。

```

                                06_http_server.cc
1  #include "common.h"
2  #include <iostream>
3  #include <workflow/HttpMessage.h>
4  #include <workflow/HttpUtil.h>
5  #include <workflow/WFFacilities.h>
6  #include <workflow/WFHttpServer.h>
7  #include <workflow/WFServer.h>
8
9  using namespace std;
10 using namespace protocol;
11
12 void process(WFHttpTask* task)
13 {
14     // 1. 获取客户端的套接字地址
15     struct sockaddr_storage addr;
16     socklen_t length = sizeof(addr);
17     task->get_peer_addr((struct sockaddr*)&addr, &length);
18     // 2. 解析套接字地址
19     char ipstr[128];
20     unsigned short port;
21     if (addr.ss_family == AF_INET) {
22         struct sockaddr_in* sin = (struct sockaddr_in*)&addr;
23         inet_ntop(AF_INET, &sin->sin_addr, ipstr, 128);
24         port = ntohs(sin->sin_port);

```

```

25     } else if (addr.ss_family == AF_INET6) {
26         struct sockaddr_in6* sin6 = (struct sockaddr_in6*)&addr;
27         inet_ntop(AF_INET6, &sin6->sin6_addr, ipstr, 128);
28         port = ntohs(sin6->sin6_port);
29     } else {
30         strcpy(ipstr, "Unknown address family");
31     }
32     cout << "客户端地址: " << ipstr << ":" << port << endl;
33     // 3. 打印请求序号 (从 0 编号)
34     long long seq = task->get_task_seq();
35     cout << "请求序号: " << seq << endl;
36     // 4. 解析请求 --> [处理业务逻辑] --> 生成响应 (根据处理结果)
37     HttpRequest* req = task->get_req();
38     HttpResponse* resp = task->get_resp();
39     // 响应行
40     resp->set_http_version("HTTP/1.1");
41     resp->set_status_code("200");
42     resp->set_reason_phrase("OK");
43     // 响应头部
44     resp->add_header_pair("Content-Type", "text/html");
45     resp->add_header_pair("Server", "My WFHttpServer");
46     if (seq == 9) {
47         // 如果是第 10 次请求, 将 Connection 设置为 close (服务器主动断开连接)
48         resp->set_header_pair("Connection", "close");
49     }
50     // 响应主体
51     resp->append_output_body_nocopy("<html>"); // 静态存储期限: 与天地同寿
52     string line;
53     line = line + "<p>" + req->get_method() + " "
54         + req->get_request_uri() + " "
55         + req->get_http_version() + "</p>";
56     resp->append_output_body(line);
57     // 遍历请求头部
58     HttpHeadersCursor cursor(req);
59     string name;
60     string value;
61     while (cursor.next(name, value)) {
62         string header = "<p>" + name + ": " + value + "</p>";
63         resp->append_output_body(header);
64     }
65     resp->append_output_body_nocopy("</html>");
66 }
67
68 static WFFacilities::WaitGroup waitGroup(1);
69
70 void sig_handler(int signo)
71 {
72     waitGroup.done();
73 }
74
75 int main(int argc, char* argv[])

```

```

76 {
77     // 1. 注册信号处理函数
78     signal(SIGINT, sig_handler);
79     // 2. 创建 HTTP 服务器
80     WFHttpServer server(process);
81     // 3. 启动服务器：绑定通配符地址，监听在 8888 端口
82     if (server.start(8888) == 0) {
83         waitGroup.wait();
84         server.stop(); // 让服务器优雅退出
85     } else {
86         cerr << "ERROR: Server start failed!" << endl;
87         exit(1);
88     }
89     return 0;
90 }

```

5.2 讲解

1. 创建 *HTTP* 服务器

在 Workflow 中，我们用 `WFHttpServer` 表示 HTTP 服务器。创建 `WFHttpServer` 时，需要传入一个 `process` 函数。故名思义，每当服务器接收到一个请求，都会调用 `process` 函数来处理这个请求。`process` 函数的类型和 `WFHttpTask` 回调函数的类型一样，都是 `std::function<void (WFHttpTask *)>`。

WFHttpServer 处理一个请求的流程如下:

1. 每当服务器接收一个请求时，Workflow 都会创建一个序列（SeriesWork）为这个请求服务。服务结束后，这个序列就会被销毁。
2. 创建 WFHttpTask 任务，并根据请求报文，初始化 WFHttpTask 的 req。并将 WFHttpTask 至于序列的末尾。
3. 调用 WFHttpServer 的 process 函数，将刚创建的 WFHttpTask 作为 process 函数的参数传入。
4. process 函数结束后，Workflow 会启动序列。

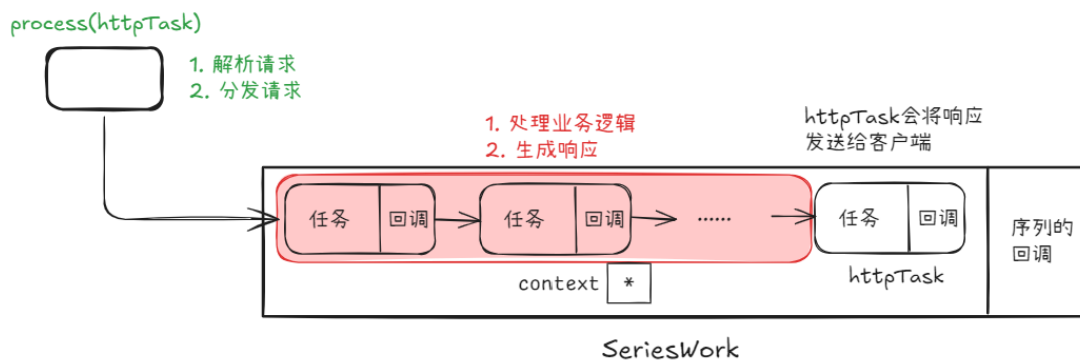


Figure 2: HTTP 服务器流程图

2. 启动和关闭服务器

我们可以调用 `server.start(port)` 来启动服务器，服务器会绑定通配符地址，并监听端口 `port`。`start()` 是非阻塞的，所以启动服务器后，我们需要用某个方式阻塞主线程。在这个例子中，我们使用的是 `waitGroup.wait()` 方法。

`server.stop()` 可以关闭服务器，且 `stop()` 是非暴力式的，它会等待所有的请求都被服务完，然后才会关闭服务器。

6 磁盘 I/O 任务

6.1 示例：静态资源服务器

这一节，我将带大家实现一个静态资源服务器。所谓静态资源，就是不会变化的资源，一般是以文件的形式存放在服务器的磁盘上。这个程序主要展示了磁盘 IO 任务的用法。

Linux 操作系统支持一套效率很高，CPU 占用非常少的异步 IO 系统调用。在 Linux 系统下使用 Workflow 框架将默认使用这套接口。

```
07_http_file_server.cc
1 #include "common.h"
2 #include <iostream>
3 #include <workflow/HttpMessage.h>
4 #include <workflow/WFFacilities.h>
5 #include <workflow/WFHttpServer.h>
6
7 using namespace std;
8 using namespace std::placeholders;
9 using namespace protocol;
10
11 void pread_callback(WFFileIOTask* task, HttpResponse* resp, string filename)
12 {
13     /*
14      * struct FileIOArgs
15      * {
16      *     int fd;
17      *     void* buf;
18      *     size_t count;
19      *     off_t offset;
20      * };
21      */
22     FileIOArgs* args = task->get_args();
23     close(args->fd); // 不要忘记关闭文件描述符!
24     // 返回已读取的字节数目, 任务已完成, 该方法不会阻塞
25     long bytes = task->get_retval();
26     if (task->get_state() != WFT_STATE_SUCCESS) {
27         resp->set_status_code("500");
28         resp->append_output_body("<html>500 Internal Server Error.</html>");
29     } else {
30         /* 慎重使用 '_nocopy' */
31         // 需要保证 args->buf 的声明周期长于 resp 所在的 WFHttpTask!
32         resp->add_header_pair("Content-Disposition", "attachment; filename=" + filename);
33         resp->append_output_body_nocopy(args->buf, bytes);
34     }
35 }
36
37 void process(WFHttpTask* task)
38 {
39     // 1. 解析请求, 获取用户要下载的文件路径和文件名
40     HttpRequest* req = task->get_req();
41     string uri = req->get_request_uri();
```

```

42 // 截取路径部分
43 size_t pos = uri.find('?');
44 string path;
45 if (pos != string::npos) {
46     path = uri.substr(0, pos);
47 } else {
48     path = uri;
49 }
50 // 如果路径为 "/", 获取首页 index.html
51 if (path == "/") {
52     path += "index.html";
53 }
54 // 将 URI 中的路径 --> 服务器文件系统的路径
55 path = "resources" + path;
56 // 获取文件名
57 string filename = path.substr(path.find_last_of('/') + 1);
58 // 打印调试信息
59 cout << "path: " << path << ", filename: " << filename << endl;
60 // 2. 设置通用的响应头部 (成功和失败都需要设置的头部)
61 HttpResponse* resp = task->get_resp();
62 resp->add_header_pair("Server", "Sogou C++ Workflow Server");
63 // 3. 创建 WFFileIOTask, 读取文件内容
64 int fd = open(path.c_str(), O_RDONLY);
65 if (fd == -1) {
66     resp->set_status_code("404");
67     resp->append_output_body("<html>404 Not Found.</html>");
68     return;
69 }
70 // 创建存放数据的 buf
71 size_t size = lseek(fd, 0, SEEK_END);
72 void* buf = malloc(size);
73 assert(buf != nullptr && "malloc failed"); // 断言: buf 不为空, 否则程序崩溃
74 // 这里我们利用任务的 user_data 来传递数据
75 // 如果是更复杂的情况, 请使用序列的上下文来传递数据
76 task->user_data = buf;
77 // free in task's callback
78 task->set_callback([](WFHttpTask* task) {
79     free(task->user_data);
80 });
81 WFFileIOTask* preadTask = WFTaskFactory::create_pread_task(
82     fd, // 文件描述符
83     buf, // 存放数据的 buf
84     size, // 最多读取的字节数目
85     0, // offset: 偏移量
86     bind(pread_callback, _1, resp, filename) // 回调函数
87 );
88 // 将 WFFileIOTask 追加到序列中
89 series_of(task)->push_back(preadTask);
90 }
91
92 static WFFacilities::WaitGroup waitGroup(1);

```



```

93
94 void sig_handler(int)
95 {
96     waitGroup.done();
97 }
98
99 int main()
100 {
101     // 1. 注册信号处理函数
102     signal(SIGINT, sig_handler);
103     // 2. 使用默认参数, 创建 HTTP 服务器
104     WFHttpServer server(process);
105     // 3. 启动 HTTP 服务器, 绑定通配符地址, 并监听在 8080 端口
106     if (server.start(8080) == 0) {
107         waitGroup.wait();
108         server.stop();
109     } else {
110         cerr << "ERROR: Server start FAILED!" << endl;
111         exit(1);
112     }
113     return 0;
114 }

```

6.2 讲解

1. 创建任务

我们可以通过任务工厂 WFTaskFactory 来创建磁盘 I/O 任务。无论是 create_pread_task() 还是 create_pwrite_task(), 返回的都是 WFFileIOTask。

```

struct FileIOArgs
{
    int fd;
    void* buf;
    size_t count;
    off_t offset;
};

...
using WFFileIOTask = WFFileTask<struct FileIOArgs>;
using fio_callback_t = std::function<void (WFFileIOTask *)>;
...

class WFTaskFactory
{
public:
    static WFFileIOTask *create_pread_task(int fd, void *buf, size_t count, off_t offset,
        fio_callback_t callback);

    static WFFileIOTask *create_pwrite_task(int fd, void *buf, size_t count, off_t offset,
        fio_callback_t callback);

```

```

    /* Interface with file path name */
    static WFFileIOTask *create_pread_task(const std::string& path, void *buf, size_t count, off_t offset,
        fio_callback_t callback);

    static WFFileIOTask *create_pwrite_task(const std::string& path, void *buf, size_t count, off_t offset,
        fio_callback_t callback);
    // ...
};

```

2. 处理结果（回调函数）

`task->get_args()` 可以获取任务的输入参数。对于 `WFFileIOTask` 任务，参数为 `FileIOArgs` 结构体。如果是用文件路径名的方式创建的 `WFFileIOTask`，`FileIOArgs` 结构体中的 `fd` 的值为 `-1`。

`task->get_retval()` 可以获取读写操作的返回值。返回值小于 0，表示 `WFFileIOTask` 任务失败，与 `task->get_state() != WFT_STATE_SUCCESS` 完全等价；否则，返回值表示实际读写的字节数目。

```

template<class ARGS>
class WFFileTask : public IORequest
{
public:
    void start();
    void dismiss();
public:
    ARGS *get_args() { return &this->args; }
    long get_retval() const;
public:
    void *user_data;
public:
    int get_state() const { return this->state; }
    int get_error() const { return this->error; }
public:
    void set_callback(std::function<void (WFFileTask<ARGS> *)> cb);
protected:
    ARGS args;
    std::function<void (WFFileTask<ARGS> *)> callback;
    // ...
};

```

7 定时器任务

7.1 示例：周期性执行任务

Workflow 将时间的流逝抽象成了定时任务：`WFTimerTask`。在这个示例中，我们将创建一个周期性执行的任务：3s 后首次执行，之后每隔 1s 执行一次。

```

08_timer_task.cc
1 #include <workflow/WFTaskFactory.h>
2
3 using namespace std;
4

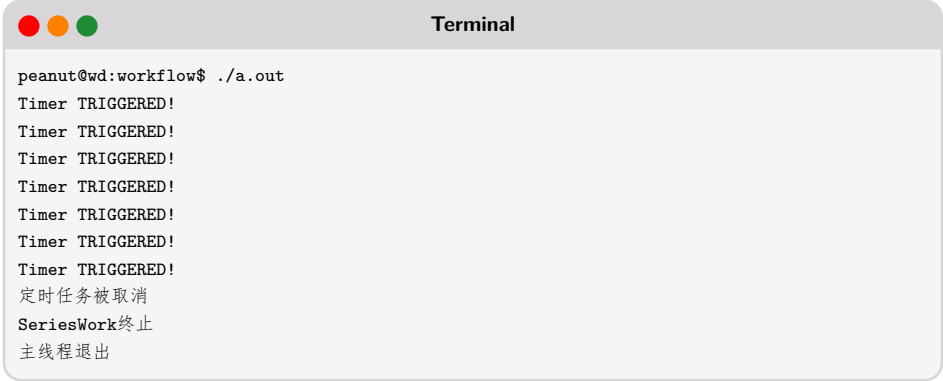
```

```

5 void timer_callback(WFTimerTask* task)
6 {
7     // 尽管任务被取消，任务的回调函数依然会执行！
8     if (task->get_state() != WFT_STATE_SUCCESS) {
9         cout << "定时任务被取消" << endl;
10        return;
11    }
12    cout << "Timer TRIGGERED!" << endl;
13    WFTimerTask* next = WFTaskFactory::create_timer_task("demo", 1, 0, timer_callback);
14    series_of(task)->push_back(next);
15 }
16
17 int main()
18 {
19     // Workflow 将时间的流逝抽象成了定时任务：WFTimerTask
20     // 1. 创建定时任务
21     WFTimerTask* task = WFTaskFactory::create_timer_task("demo", 3, 0, timer_callback);
22     // 2. 将定时任务放入序列中，并启动序列
23     WFFacilities::WaitGroup waitGroup(1);
24     SeriesWork* series = Workflow::create_series_work(task, [&waitGroup](const SeriesWork*) {
25         cout << "SeriesWork 终止" << endl;
26         waitGroup.done();
27     });
28     series->start();
29     // 3. 主线程休眠 10s
30     sleep(10);
31     // 4. 取消任务
32     WFTaskFactory::cancel_by_name("demo");
33     // 5. 主线程等待序列终止
34     waitGroup.wait();
35     cout << "主线程退出" << endl;
36     return 0;
37 }

```

运行结果：



```

Terminal
peanut@wd:workflow$ ./a.out
Timer TRIGGERED!
Timer TRIGGERED!
Timer TRIGGERED!
Timer TRIGGERED!
Timer TRIGGERED!
Timer TRIGGERED!
Timer TRIGGERED!
定时任务被取消
SeriesWork终止
主线程退出

```

7.2 讲解

定时器任务会不占用线程地等待一个确定时间，当该时间一过，Workflow 框架就会调用定时器任务的回调函数。

1. 创建任务和取消任务

WTFTaskFactory 工厂类中有四个和定时器任务相关的接口：

```
WTFTaskFactory.h
using timer_callback_t = std::function<void (WFTimerTask *)>;

class WTFTaskFactory
{
public:
    // 创建匿名的定时器任务
    static WFTimerTask *
    create_timer_task(time_t seconds, long nanoseconds, timer_callback_t callback);

    // 创建有名字的定时器任务
    static WFTimerTask *
    create_timer_task(const std::string& timer_name, time_t seconds, long nanoseconds, timer_callback_t callback);

    // 取消 timer_name 名字下的所有定时器
    static int cancel_by_name(const std::string& timer_name)
    {
        cancel_by_name(const std::string& timer_name, (size_t)-1);
    }

    // 取消 timer_name 名字下的最多 max 个定时器
    static int cancel_by_name(const std::string& timer_name, size_t max);
};
```

我们可以创建匿名定时器，也可以在创建定时器时给它指定一个名字，方便以后调用 `cancel_by_name()` 来取消定时器。当 `seconds` 参数的值为 `-1` 时，Workflow 会创建一个无限时长的定时器。（思考：无限时长的定时器有什么用？）

`cancel_by_name()` 可以取消定时器。注意：尽管定时器任务被取消了，它的回调函数依然会执行。

8 计算任务

8.1 示例：WFGoTask

Workflow 有多种方式创建计算任务，官方最推荐、也最容易使用的是 `WFGoTask`。`WFGoTask` 是模仿 go 语言协程实现的，它允许你将一个普通的 C++ 函数（如一个耗时的计算任务）封装成一个 Task 对象，然后交给 Workflow 的后台线程池去异步执行。

```
09_go_task.cc
1 #include "common.h"
2 #include <iostream>
3 #include <workflow/WFFacilities.h>
4 #include <workflow/WFTaskFactory.h>
5
6 using namespace std;
7
8 void add(int a, int b, int& result)
9 {
10     // 模拟耗时的计算
11     sleep(3);
12     result = a + b;
13 }
```

```

14
15 int main()
16 {
17     // 1. 创建 WFGoTask
18     int a = 1, b = 2;
19     int result;
20     // ERROR: 参数是值传递的
21     // WFGoTask* task = WFTaskFactory::create_go_task("queue1", add, a, b, result);
22     WFGoTask* task = WFTaskFactory::create_go_task("queue1", add, a, b, ref(result));
23     // 2. 设置 WFGoTask 任务的回调函数
24     WFFacilities::WaitGroup waitGroup(1);
25     task->set_callback([&](WFGoTask* task) {
26         cout << a << " + " << b << " = " << result << endl;
27         cout << "WFGoTask 执行完毕" << endl;
28         waitGroup.done();
29     });
30     // 3. 启动任务: 交给框架异步执行
31     task->start();
32     // 4. 主线程等待任务执行完毕
33     waitGroup.wait();
34     return 0;
35 }

```

运行结果:



```

Terminal
peanut@wd:workflow$ ./a.out
1 + 2 = 3
WFGoTask 执行完毕

```

8.2 讲解

1. 创建任务

```

WFTaskFactory.h
class WFTaskFactory
{
public:
    template<class FUNC, class... ARGS>
    static WFGoTask *create_go_task(const std::string& queue_name, FUNC&& func, ARGS&&... args);
    // ...
};

```

- queue_name: 队列名。
- func: 要执行的计算任务。可以是任意的可调用对象: 函数指针, 函数对象, lambda 表达式等。
- args: 可变长参数, func 的参数。

小结:

1. 创建 WFGoTask 和创建其它任务没有太大的区别, 唯一的不同点: 创建 WFGoTask 时不需要传入回调函数 (Why?)。但和其它任务一样, 我们可以通过 task->set_callback() 来设置 WFGoTask 的回调函数。

2. 如果 `WFGoTask` 的某个参数是引用，则必须使用 `std::ref` 包装，否则就是值传递。

2. 队列名

在 `Workflow` 中，计算任务没有优先级的概念，唯一可以影响调度顺序的就是计算任务的队列名。

- 当计算线程没有 100% 占满，所有任务都是实时调起的，队列名没有任何影响。
- 当计算线程被占满的情况下，同一个队列名下的任务，它们被调度的顺序是与任务提交的顺序一致的。

总而言之，除非机器的计算负载非常繁重，否则没有必要关心队列名。

3. 把 `Workflow` 当作线程池

如果我们只使用 `WFGoTask`，那么 `Workflow` 就退化成一个线程池，默认情况下，线程数量等于机器 CPU 的核数。但是这个线程池比一般的线程池有更多的功能，比如：每个任务有队列名，任务之间还可以组成各种串并联或更复杂的关系。

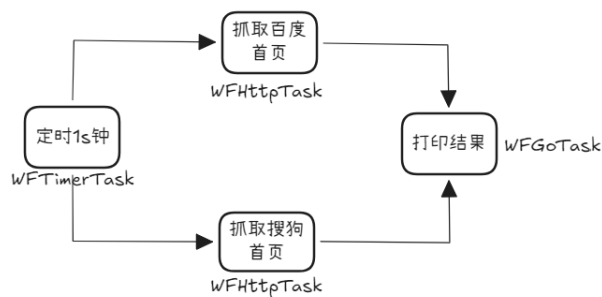
`WFGoTask` 是 `Workflow` 为用户提供的、通往异步世界的便捷通道。它用最小的代价，将普通的计算任务包装成可编排的“任务流积木”，让你在享受异步高性能的同时，不必被复杂的线程管理所困扰。

9 DAG 图任务 (了解)

9.1 示例: `WFGraphTask`

任务之间除了串联、并联之外，有时候还需要表示更加复杂的依赖关系。这时候，`WFGraphTask` 就派上用场了。

我们通过一个具体示例来演示 `WFGraphTask` 的用法：先执行一个 1 秒的定时器任务，然后并行抓取百度和搜狗的首页，待两者都完成后，再执行一个打印结果的计算任务。示意图如下：



10_graph_task.cc

```
1 #include <iostream>
2 #include <workflow/WFFacilities.h>
3 #include <workflow/WFTaskFactory.h>
4
5 using namespace std;
6
7 void http_callback(WFHttpTask* task)
8 {
9     size_t* size = static_cast<size_t*>(task->user_data);
10    int state = task->get_state();
11    if (state != WFT_STATE_SUCCESS) {
12        *size = (size_t)-1;
13        return;
14    }
15    const void* body;
16    task->get_resp()->get_parsed_body(&body, size);
17 }
```

```

18
19 int main()
20 {
21     // 1. 创建匿名定时任务: 定时 1s 钟
22     WFTimerTask* timer = WFTaskFactory::create_timer_task(3, 0, [](WFTimerTask*) {
23         cout << "timer task complete(3s)." << endl;
24     });
25     // 2. 创建 HTTP 任务: 获取 baidu 首页
26     WFHttpTask* fetch_baidu = WFTaskFactory::create_http_task("https://www.baidu.com", 3, 3, http_callback);
27     size_t size1 = 0;
28     fetch_baidu->user_data = &size1;
29     // 3. 创建 HTTP 任务: 获取 sogou 首页
30     WFHttpTask* fetch_sogou = WFTaskFactory::create_http_task("https://www.sogou.com", 3, 3, http_callback);
31     size_t size2 = 0;
32     fetch_sogou->user_data = &size2;
33     // 4. 创建计算任务: 打印结果
34     WFGoTask* display = WFTaskFactory::create_go_task("display", [&]() {
35         cout << "百度首页大小: " << size1 << endl;
36         cout << "搜狗首页大小: " << size2 << endl;
37     });
38     // 5. 创建图任务: 管理任务之间的依赖关系
39     WFFacilities::WaitGroup waitGroup(1);
40     WFGraphTask* graph = WFTaskFactory::create_graph_task([&waitGroup](WFGraphTask*) {
41         cout << "Graph task complete. Wakeup main thread." << endl;
42         waitGroup.done();
43     });
44     // 将各类任务包装成图任务的节点 (node)
45     WFGraphNode& a = graph->create_graph_node(timer);
46     WFGraphNode& b = graph->create_graph_node(fetch_baidu);
47     WFGraphNode& c = graph->create_graph_node(fetch_sogou);
48     WFGraphNode& d = graph->create_graph_node(display);
49     // 指定任务之间的依赖关系
50     a --> b;
51     a --> c;
52     b --> d;
53     c --> d;
54     // 6. 启动图任务
55     graph->start();
56     // 7. 主线程等待图任务结束
57     waitGroup.wait();
58     cout << "主线程退出" << endl;
59     return 0;
60 }

```

运行结果:

```
Terminal
peanut@wd:workflow$ ./a.out
timer task complete(3s).
百度首页大小: 29506
搜狗首页大小: 13454
Graph task complete. Wakeup main thread.
主线程退出
```

9.2 讲解

使用 WFGraphTask 非常直观，主要分为四步：创建图任务、添加节点、定义依赖、执行图任务。

1. 创建图任务

图任务也是一种任务，我们也可以通过任务工厂 WFTaskFactory 创建：

```
----- WFTaskFactory.h -----
using graph_callback_t = std::function<void (WFGraphTask *)>;

class WFTaskFactory
{
public:
    static WFGraphTask *create_graph_task(graph_callback_t callback);
    // ...
};
```

创建图任务，我们只需要传递图任务的回调函数即可。很显然我们创建的是一张空图，它没有任何的节点。

2. 添加节点我们可以调用 create_graph_node() 函数，给图任务添加节点。

```
WFGraphNode& a = graph->create_graph_node(timer);
WFGraphNode& b = graph->create_graph_node(httpTask1);
WFGraphNode& c = graph->create_graph_node(httpTask2);
WFGraphNode& d = graph->create_graph_node(goTask);
```

该函数可以把任意类型的任务包装成图的节点（装饰器模式，Wrapper），并返回该节点的引用。用户通过这个节点的引用来建立节点之间的依赖关系。如果我们只添加节点，而不建立节点之间的依赖关系，那么所有节点都是孤立节点。如果这时候启动图任务，那么所有的节点将全部并发执行！

3. 定义依赖

我们可以使用非常形象的运算符 --> 来建立节点之间的依赖关系：

```
a-->b;
a-->c;
b-->d;
c-->d;
```

很显然，Workflow 是利用 C++ 的运算符重载来实现这个功能的：

```
----- WFGraphTask.h -----
static inline WFGraphNode& operator > (WFGraphNode& prec, WFGraphNode& succ)
{
    prec.precede(succ);
    return succ;
}
```



```

static inline WFGGraphNode& operator < (WFGGraphNode& succ, WFGGraphNode& prec)
{
    succ.succeed(prec);
    return prec;
}

static inline WFGGraphNode& operator --(WFGGraphNode& node)
{
    return node;
}

```

从上面的源代码可以看得知，除了-->运算符，Workflow 还支持<--运算符，并且它们还可以连着写。所以，下面程序都是合法且等价的：

```

{
    a-->b-->d;
    a-->c-->d;
}

```

```

{
    d<--b<--a;
    d<--c<--a;
}

```

```

{
    d<--b<--a-->c-->d;
}

```

4. 执行图任务

图任务和其它任务没有区别，我们可以

- 直接运行：graph->start()。
- 将图任务放入任务流（SeriesWork, ParallelWork）中执行。
- 甚至，将图任务作为另一个图任务的节点，也是完全正确的行为。

10 ParallelWork：并行任务流

在之前的示例里，我们已经接触过了 SeriesWork 类。SeriesWork 代表一系列串行执行的任务。序列中所有任务结束，SeriesWork 才结束。

与 SeriesWork 对应的是 ParallelWork，它由 SeriesWork 构成，代表若干个 SeriesWork 并行执行。当所有的 SeriesWork 都完成，ParallelWork 才算结束。见 Figure 3:

10.1 示例：并行执行两个序列

我们首先来看一个简单的示例，它演示了 ParallelWork 的基本用法：

```

11_parallel_work.cc
1 #include <iostream>
2 #include <workflow/WFFacilities.h> // for WFFacilities::WaitGroup
3 #include <workflow/WFTaskFactory.h>

```

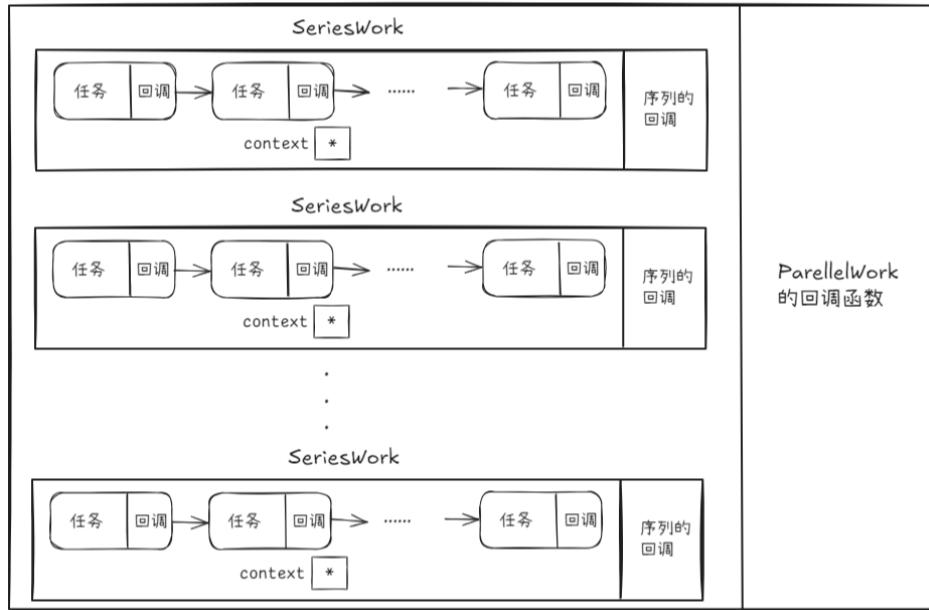


Figure 3: ParallelWork 示意图

```

4 #include <workflow/Workflow.h> // for ParallelWork
5
6 using namespace std;
7
8 int main()
9 {
10     // 1. 创建 SeriesWork* series1, 并加入三个 task
11     // series1: task1 -> task2 -> task3;
12     WFGoTask* task1 = WFTaskFactory::create_go_task("series1", []() {
13         cout << "series1: task 1 done." << endl;
14     });
15     WFGoTask* task2 = WFTaskFactory::create_go_task("series1", []() {
16         cout << "series1: task 2 done." << endl;
17     });
18     WFGoTask* task3 = WFTaskFactory::create_go_task("series1", []() {
19         cout << "series1: task 3 done." << endl;
20     });
21     SeriesWork* series1 = Workflow::create_series_work(task1, [](const SeriesWork* series) {
22         cout << "series1: done!" << endl;
23     });
24     *series1 << task2 << task3;
25     // 2. 创建 SeriesWork* series2, 并加入两个 job.
26     // series2: job1 -> job2;
27     WFGoTask* job1 = WFTaskFactory::create_go_task("series2", []() {
28         cout << "series2: job 1 done." << endl;
29     });
30     WFGoTask* job2 = WFTaskFactory::create_go_task("series2", []() {
31         cout << "series2: job 2 done." << endl;
32     });
33     SeriesWork* series2 = Workflow::create_series_work(job1, [](const SeriesWork* series) {
34         cout << "series2: done!" << endl;

```

```

35     });
36     series2->push_back(job2);
37     // 3. 创建 ParallelWork, 并将 series1 和 series2 加入其中
38     WFFacilities::WaitGroup waitGroup(1);
39     ParallelWork* parallel = Workflow::create_parallel_work([&waitGroup](const ParallelWork* work) {
40         cout << "ParallelWork done!" << endl;
41         waitGroup.done();
42     });
43     parallel->add_series(series1);
44     parallel->add_series(series2);
45     // 4. 启动 ParallelWork, 交给框架并行执行里面的序列
46     parallel->start();
47     // 5. 主线程等待 ParallelWork 完成
48     waitGroup.wait();
49     cout << "主线程退出" << endl;
50     return 0;
51 }

```

运行结果:



```

Terminal
series1: task 1 done.
series2: job 1 done.
series1: task 2 done.
series2: job 2 done.
series2: done!
series1: task 3 done.
series1: done!
ParallelWork done!
主线程退出

```

每次运行结果可能都不相同（并发执行），但：

- series1 和 series2 里面的任务是按序执行的。
- 并且 series1 和 series2 结束后，才会执行 ParallelWork 的回调函数，输出“ParallelWork done!”。
- 主线程会等待 ParallelWork 结束，因此最后才输出“主线程退出”。

10.2 示例：并行抓取网页，并按序输出

接下来，我们来看一个稍微难一捏捏的例子，类似 `wget <url>... -q -O -:`

程序从命令行读取多个 URL，并行抓取这些网页，并按用户输入 URL 的顺序输出对应的网页。如果抓取失败，则打印出错原因。

```

12_parallel_wget.cc
1 #include <iostream>
2 #include <workflow/WFFacilities.h>
3 #include <workflow/WFTaskFactory.h>
4 #include <workflow/Workflow.h>
5
6 using namespace std;
7 using namespace protocol;
8
9 // state, error: 出错原因

```

```

10 // resp: 抓取的网页
11 struct SeriesContext {
12     string url;
13     int state;
14     int error;
15     HttpResponse resp;
16 };
17
18 void http_callback(WFHttpTask* task)
19 {
20     // 在 HTTP 任务的回调函数中, 设置 SeriesContext 的 state, error 和 resp。
21     SeriesContext* ctx = static_cast<SeriesContext*>(series_of(task)->get_context());
22     ctx->state = task->get_state();
23     ctx->error = task->get_error();
24     // http_callback 执行完后, HTTP 任务就会被销毁,
25     // 其中的 HttpRequest 和 HttpResponse 会一并销毁!
26     // 所以我们这里可以用移动语义, 避免不必要的复制
27     ctx->resp = move(*task->get_resp());
28 }
29
30 void parallel_callback(const ParallelWork* parallel)
31 {
32     for (int i = 0; i < parallel->size(); ++i) {
33         const SeriesWork* series = parallel->series_at(i);
34         SeriesContext* ctx = static_cast<SeriesContext*>(series->get_context());
35         // 打印 URL
36         cout << ctx->url << ": " << endl;
37         if (ctx->state == WFT_STATE_SUCCESS) {
38             // 成功, 打印网页内容
39             const void* body;
40             size_t size;
41             ctx->resp.get_parsed_body(&body, &size);
42             cout << static_cast<const char*>(body) << "\n\n";
43         } else {
44             // 失败, 打印出错信息
45             cout << WFGlobal::get_error_string(ctx->state, ctx->error) << endl;
46         }
47         // 上下文使用完毕, 不要忘记销毁!
48         delete ctx;
49     }
50 }
51
52 int main(int argc, char* argv[])
53 {
54     if (argc < 2) {
55         cout << "Usage: " << argv[0] << "<url>..." << endl;
56         exit(1);
57     }
58     // 1. 创建空的并行任务
59     ParallelWork* parallel = Workflow::create_parallel_work(parallel_callback);
60     // 2. 读取用户输入的 URL, 创建序列, 并添加到 ParallelWork 中。

```

```

61     for (int i = 1; i < argc; ++i) {
62         // 创建 HTTP 任务
63         WFHttpTask* task = WFTaskFactory::create_http_task(argv[i], 3, 3, http_callback);
64         // 设置请求
65         HttpRequest* req = task->get_req();
66         req->add_header_pair("Accept", "/*/*");
67         req->add_header_pair("User-Agent", "paralle_wget (linux-gnu)");
68         req->add_header_pair("Connection", "close");
69         // 创建序列，并将 HTTP 任务添加到序列中
70         SeriesWork* series = Workflow::create_series_work(task, nullptr);
71
72         // 题目要求按输入 URL 的顺序打印对应的网页，但 ParallelWork 中的序列是并发执行的。
73         // 因此，我们不能在序列的回调函数中直接输出抓取的网页，
74         // 而应该在 ParallelWork 的回调函数中，按输入 URL 的顺序依次输出。
75         // (回顾: ParallelWork 的回调函数会在所有序列执行完后才执行)
76         // 所以我们需要在序列的上下文中保存抓取的结果。
77         SeriesContext* ctx = new SeriesContext;
78         // 此时，HTTP 任务还没执行，我们只需要设置 ctx->url 即可
79         // ctx->state, ctx->error, ctx->resp 需要在 HTTP 任务的回调函数中设置
80         ctx->url = argv[i];
81         series->set_context(ctx);
82         parallel->add_series(series);
83     }
84     // 3. 将 ParallelWork 放入一个序列中执行 (ParallelWork 也是一个任务)。
85     // 在 Workflow 中，这是一种更加规范的写法。
86     WFFacilities::WaitGroup waitGroup(1);
87     Workflow::start_series_work(parallel, [&waitGroup](const SeriesWork*) {
88         waitGroup.done();
89     });
90     // 4. 主线程等待 ParallelWork 结束
91     waitGroup.wait();
92     return 0;
93 }

```

10.3 讲解

1. 创建并行任务流: ParallelWork

在 Workflow 类里，有两个接口用于创建并行任务流：

```

class Workflow
{
public:
    static ParallelWork *
    create_parallel_work(parallel_callback_t callback);    // 创建空的并行任务流

    static ParallelWork *
    create_parallel_work(SeriesWork *const all_series[], size_t n, parallel_callback_t callback); // 创建包含 n 个序列的任务流
    // ...
};

```

无论用哪个接口创建并行任务流，在启动之前，都可以调用 `add_series()` 往并行任务流中添加新的序列。

2. 并行任务流也是任务

并行任务流也是一种任务，我们可以将它放入序列或其它的并行任务流中；甚至我们可以将并行任务流作为图任务(`WFGGraphTask`)的一个节点。这样我们就可以构建任意复杂的任务关系了。

```
Workflow.h
class ParallelWork : public ParallelTask
{
public:
    void start();
    void dismiss();
public:
    void add_series(SeriesWork *series);
public:
    void *get_context() const { return this->context; }
    void set_context(void *context) { this->context = context; }
public:
    SeriesWork *series_at(size_t index);
    const SeriesWork *series_at(size_t index) const;
    SeriesWork& operator[] (size_t index);
    const SeriesWork& operator[] (size_t index) const;
    size_t size() const;
public:
    void set_callback(parallel_callback_t callback);
    // ...
};
```

3. 总结

用好 Workflow 的关键点

从上面的例子中，我们可以看出，用好 Workflow，核心在于以下两点：

1. 想好业务流程（这也是 Workflow 名字的由来），构建对应的任务流（`SeriesWork`、`ParallelWork` 或 `WFGGraphTask`）。
2. 再写各个任务的回调函数。回调函数怎么写，很大程度上取决于任务之间如何传递数据。